



Weaver

0.3.0 Documentation
© Modula, 2025

Table Of Contents

1. [Weaver in cPanel](#)
2. [A simple Application](#)
3. [Processing the Request](#)
4. [Application Routing](#)
5. [User IDs](#)
6. [Honeypotting Bad Requests](#)

Weaver in cPanel

We recommend using `git` to manage your projects, and the instructions herein will include some helpful instructions on building your project so that it's as easy as possible to build and update your website using the `git` client.

Make sure that the `Git™ Version Control` is enabled in your cPanel. Initialize your server-side app directory by clicking `Create`, make sure `Clone a Repository` is toggled off, then set the `Repository Path` to wherever you want the source files to physically exist on the server. After doing this, click the `Manage` button next to the newly created `Git Repository` and copy the `Clone URL`. On your local development machine, you'll want to `git clone` the url you just copied to link your local development directory with the service running on the server.

After making sure the `Setup Node.js App` tool is also enabled in your cPanel, Click on the `Create Application` button and select the latest version of Node (currently `22.8.0`). Set the `Application Mode` to `Production`, point `Application Root` to the path you set the server-side app directory earlier, and name your `Application Startup File`.

Once you've setup both the `Git Repo` and the `Node app`, you'll want to run `git pull` on your local repository, which should give you a default `app.js` (or whatever you named it) you chose in the last step. Download the `weaver.js` source file from modula.dev/weaver.js in the local directory, and then include it like

```
const weaver = require("./weaver.js");
```

You'll need to define a function to handle the routing of your application, and then tell `weaver` to use it to handle incoming requests by passing it into the `weaver.router` function. And after all the other application logic, you'll call `weaver.listen`. Do not pass the `port` argument into `weaver.listen` when using it on cPanel.

A simple application

First, make sure to download and include the Weaver source like

```
const weaver = require("./weaver.js")
```

Weaver applications need to define a router – a function which takes in a request object, and a handler function, and will respond to the request using the `weaver.respond` function.

The `weaver.respond` function expects to be given an object that contains the HTTP code we're sending back, the mime type of the data we're going to return, and the body containing the content of our response. Weaver also gives us a few helper functions like `weaver.serveFile` and `weaver.serveRedirect` which can create these objects for us, but for now we're going to make the response object ourselves.

Let's define a `main` function that we will use as our router. We're just going to do a simple "Hello World" for now:

```
function main(request, handler) {  
  return {  
    code: 200,  
    mime: "text/raw",  
    body: "Hello, world!\n"  
  }  
}
```

To tell Weaver we want this to be our router function, we just pass like

```
weaver.router(main)
```

and then wrap up by calling the listener like

```
weaver.listen()
```

Processing the Request

Weaver also gives us some helper functions to processing incoming requests.

`weaver.hashId` will take in the request and turn the headers into a unique integer for each client/user-agent connecting to your service. In cases where there is not enough information in the headers to do so, this function will return undefined.

`weaver.routeMethod` will take in the request and return the method the client is using, such as GET.

`weaver.routeUri` will take in the request and return just the url part with any queries stripped out.

`weaver.routeQueryObject` will take in the request and return a dictionary/object of the key-value pairs of the query.

`weaver.routeMatch` will match the request url against a list of string regexes, and tell us which was the first that matched given a set of flags. Generally, we recommend just setting flags to "i" (case-insensitive matches). If you need help making regexes, the tool regex101.com is incredible.

Application Routing

For apps that are a little more complex than a Hello World, you're going to want to define your router so that it processes incoming requests, and then passes them off to appropriate functions for each route.

Let's make a simple router that gives any requests for the root to function `main` which returns an `index.html`, any requests for the subdirectory `bar` to the function `foo` which will redirect, and everything else to the function `bad`.

Our `app.js` now will look something like

```
const weaver = require("./weaver.js");
function main() { return weaver.sendFile("text/html", "index.html"); }
function foo() { return weaver.redirect("/"); }
function bad() { return {
  code: 404,
  mime: "text/raw",
  body: "HTTP 404: Route does not exist" }
}
const foo_route = "^(https?:\\/\n)?[^\n]*\n/bar(\\[^\n]*)*$";
const main_route = "^(https?:\\/\n)?[^\n]*(\\/\n)?$";
function router(request, handler){
  const path = weaver.routeUri(request);
  const routes = [foo_route, main_route];
  const use = weaver.routeMatch(request, routes);
  var response;
  switch(use) {
    case 0: response = foo(); break;
    case 1: response = main(); break;
    default: response = bad(); break;
  }
  weaver.respond(handler, response);
}
weaver.router(router)
weaver.listen()
```

User IDs

It's very likely you're not just serving static content, but instead that you want to provide some service to remote users over the internet. To do that, you usually need to keep track of who made a given request.

We personally believe that fingerprinting users is invasive and impolite, and we don't like asking users to keep cookies or other identifying marks on their system, so instead Weaver provides a very simple `hashID` function that gives users a unique number ID based on their self-reported `user-agent` and connecting IP address, meaning they're temporary and non-identifying but still usable for things like allowing remote clients to log into a service.

To get this unique ID, all we have to do is pass the `request` into `hashID` somewhere in our app's router like so

```
function router(request, handler){
  const client = weaver.hashId(request);
  // router logic
}
```

Since the `app.js` is run independently between connections, you'll likely want to store what IDs correspond to what user somewhere on disk, and then have those sessions expire either on a timer or when that user logs in another session.

Honeypotting Bad Requests

In our previous example, we assumed all incoming requests were valid and genuine, but any internet-connected service is inevitably going to receive malicious traffic. While Weaver does not provide any anti-malware or security services directly, it does provide a very minimal anti-spam feature using honeypots.

We can modify our previous router function by adding in a quick filter and check like so like so

```
function router(request, handler){
  const path = weaver.routeUri(request);
  const routes = [foo_route, main_route];
  const filter = weaver.routeHoneypot(request);
  if (filter !== undefined) { response = filter; }
  else {
    const use = weaver.routeMatch(request, routes);
    var response;
    switch(use) {
      case 0: response = foo(); break;
      case 1: response = main(); break;
      default: response = bad(); break;
    }
  }
  weaver.respond(handler, response);
}
```

The `routeHoneypot` function either returns the response for a 404: Resource Not Found if the user is attempting to access something in our honeypot, or it returns `undefined` in which case we carry out with our normal routing logic.

We could go a step farther and add that user-agent to a blacklist and serve all incoming requests from them going forward with that same 404 error, if we wished, but we'll leave that as an exercise for you.

You can optionally add additional regexes to the honeypot using the `pushHoneypot` function.